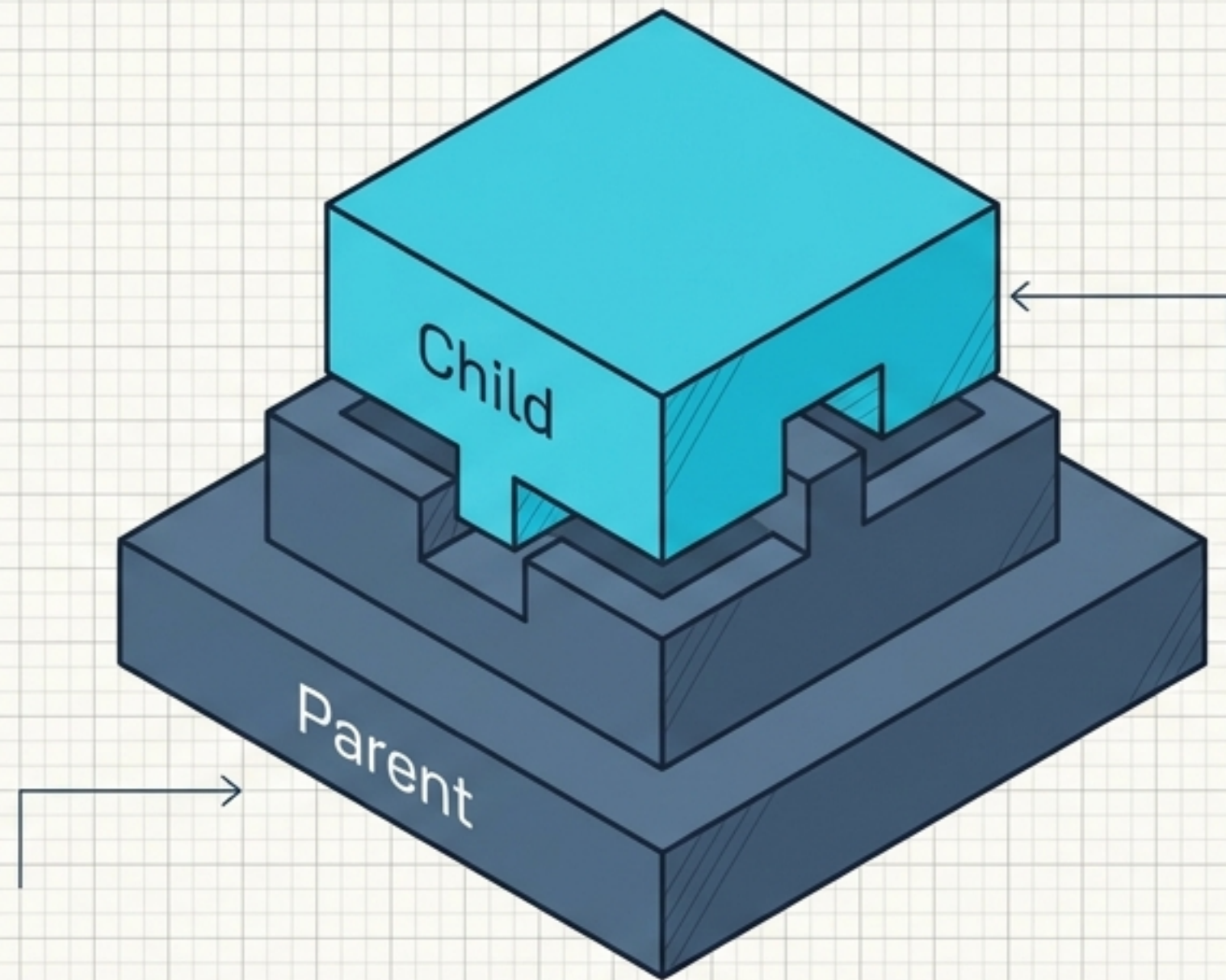
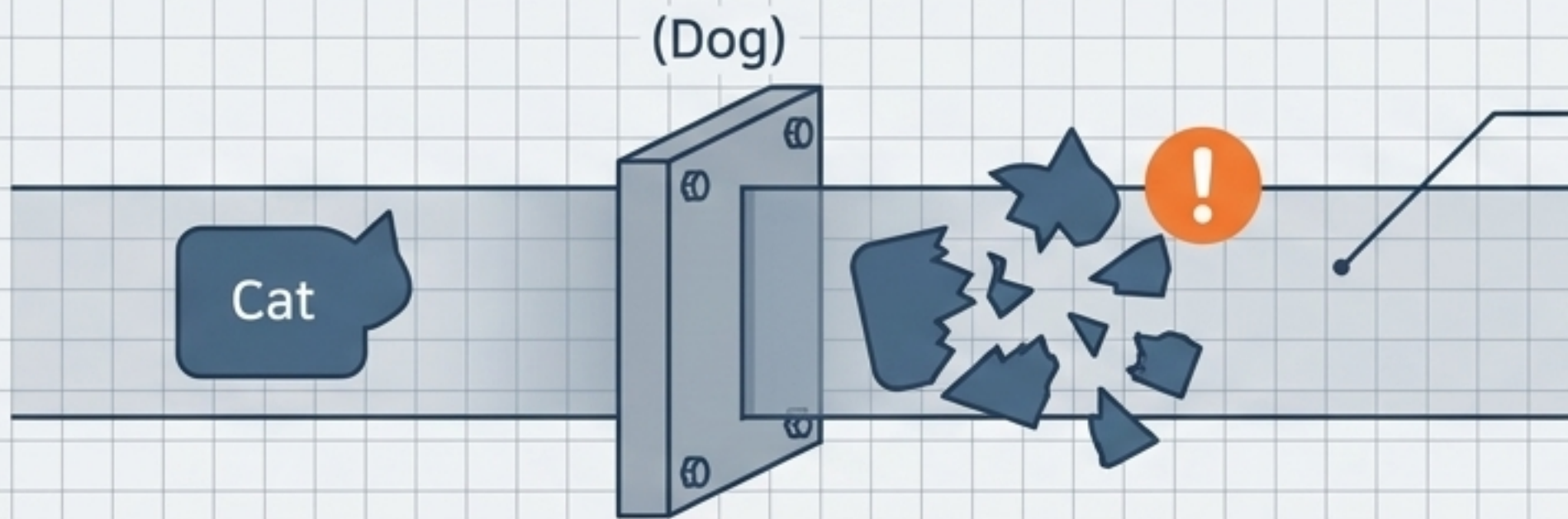


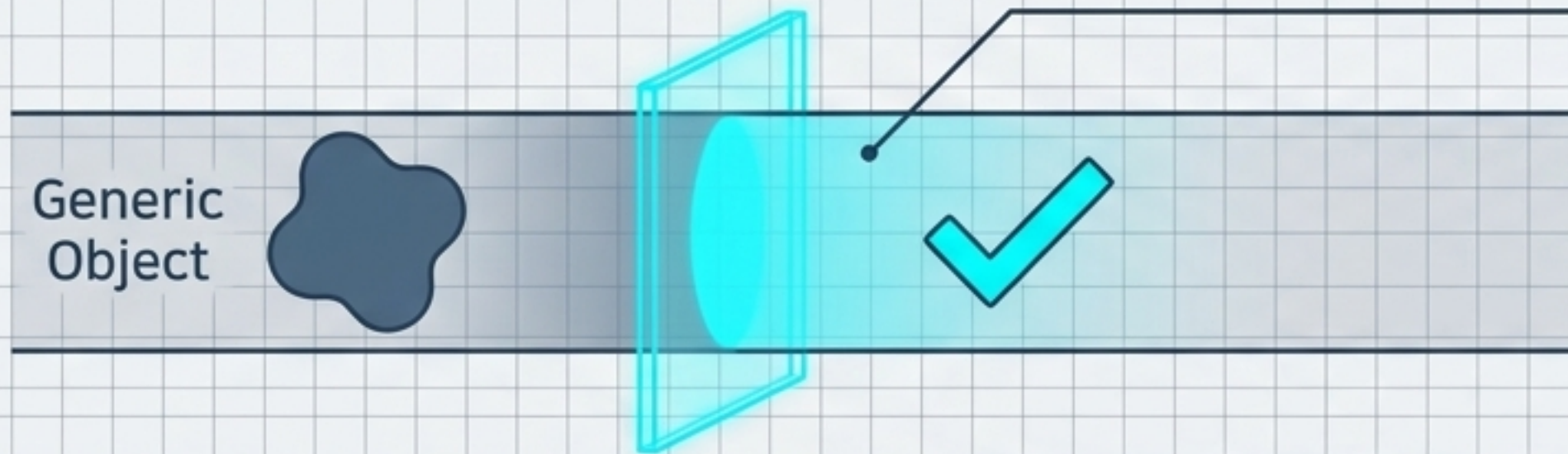


상속과 다형성의 완성

C# 객체 지향 아키텍처 완벽 제어 가이드



일반 캐스팅: 실패 시 치명적인 예외 발생



is 키워드: 객체의 타입만 확인 (True/False)



as 키워드: 안전한 변환 시도.
실패 시 **null** 반환.
프로그램 중단 없음.

```
IDE
변수 as 클래스
```

안티 패턴

```
if (item is Dog) {  
    (item as Dog).Bark();  
}
```

→ is로 이미 확인했는데 as를 다시 사용하는 것은 낭비. as 결과가 null일 가능성이 이미 0%이기 때문.

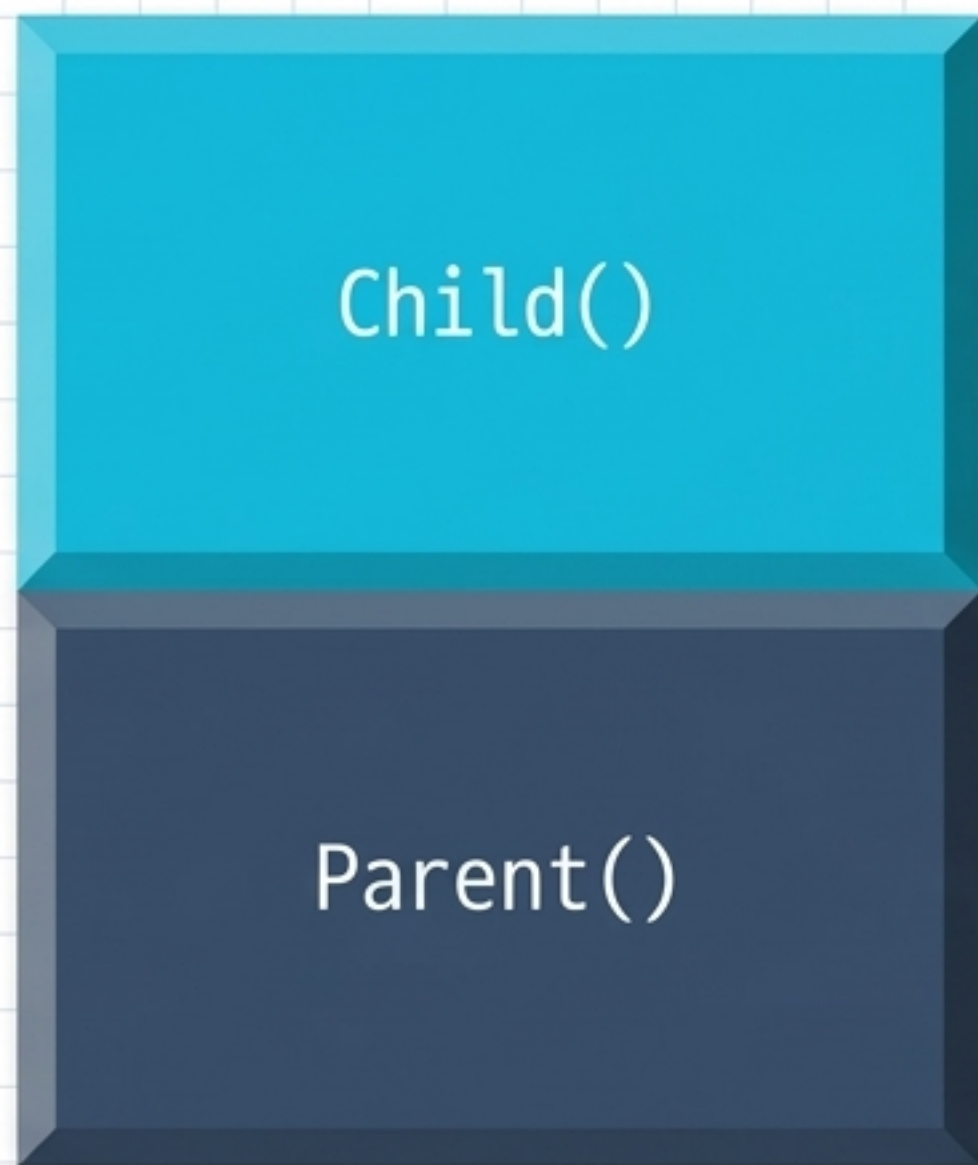
권장 패턴

```
var dog = item as Dog;  
if (dog != null) {  
    dog.Bark();  
}
```

→ 형변환을 한 번만 수행하고 null 체크로 안전하게 실행. 가장 권장되는 형태.

생성자 스택 (Constructor Stack)

실행 순서



자식 인스턴스를 생성하면 부모 멤버 초기화를 위해 부모 생성자가 항상 먼저 호출됩니다.
기반(Parent) 없이는 층(Child)을 올릴 수 없습니다.

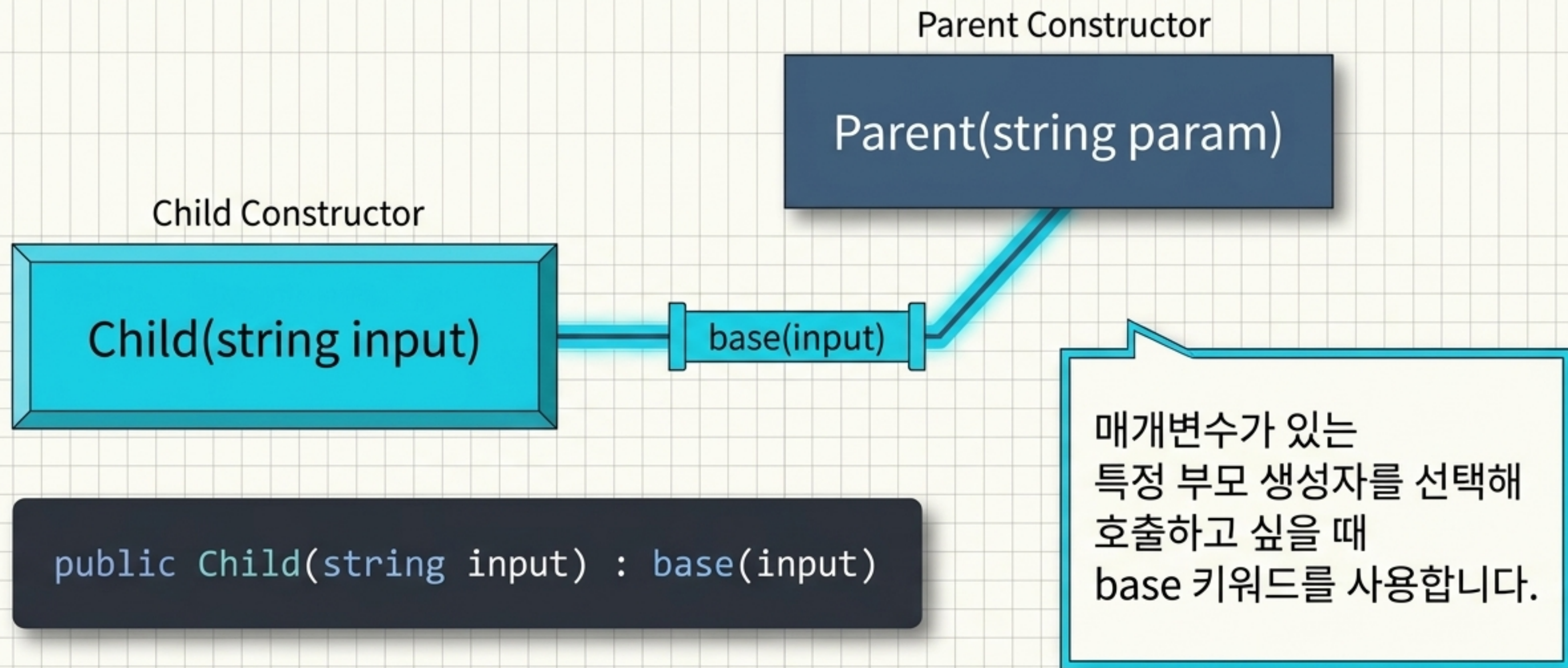
이후 자식 생성자 호출

부모 생성자
자동 호출

```
Child child = new Child();
```

부모 생성자
자식 생성자

명시적 기반 호출 (base 키워드)



클래스 변수 메모리 공유의 함정

Parent.counter

Child.counter



static int counter = 2

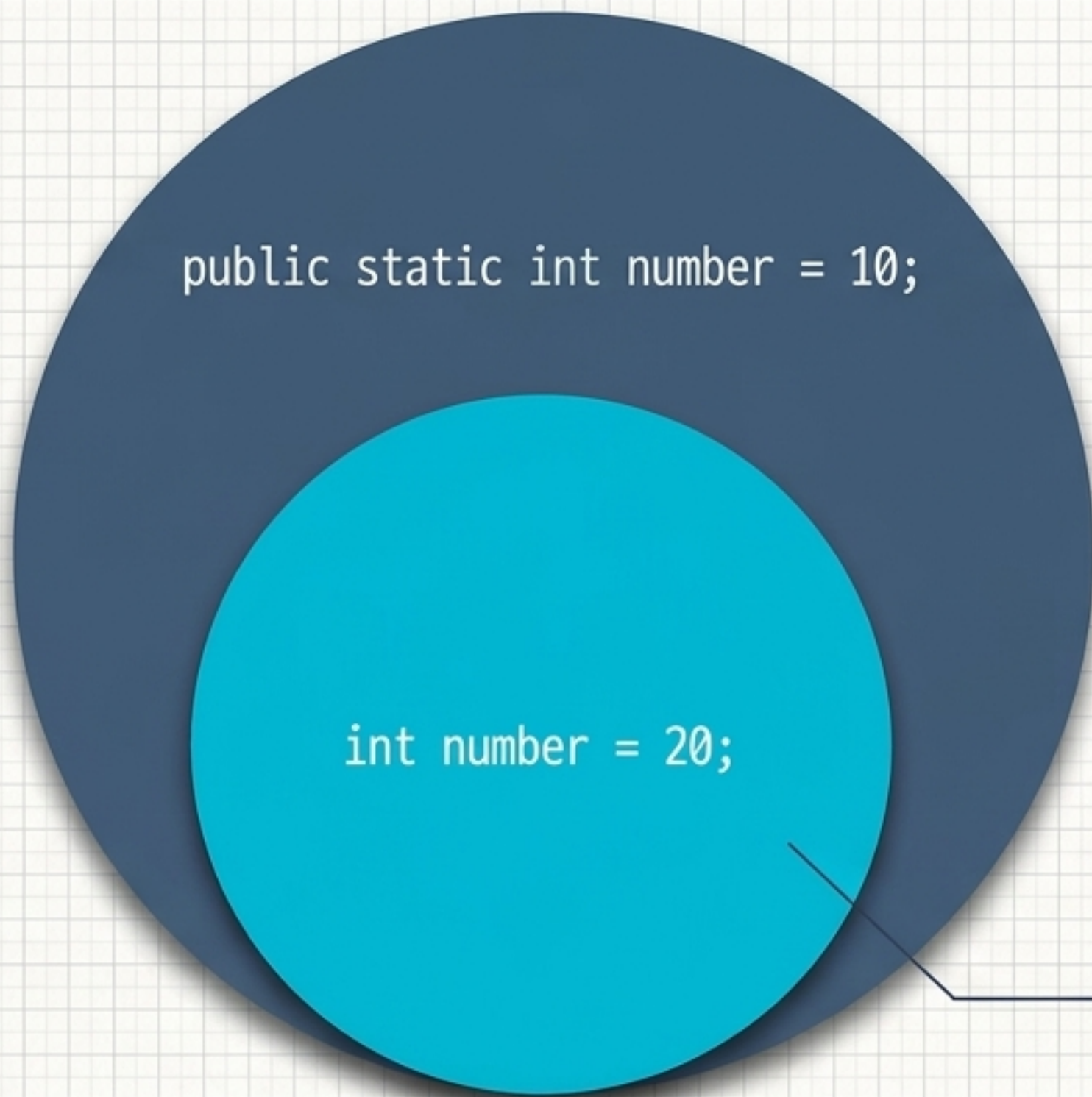
```
class Parent {  
    public static int counter = 0;  
    public void CountParent() {  
        Parent.counter++;  
    }  
}  
class Child : Parent {  
    public void CountChild() {  
        Child.counter++;  
    }  
}
```

static 변수는 상속되더라도 각 클래스마다 따로 생성되지 않습니다.
부모와 자식이 정확히 동일한 메모리 공간을 공유합니다.

실행 결과

2
2

스코프 충돌과 섀도잉 (Shadowing)



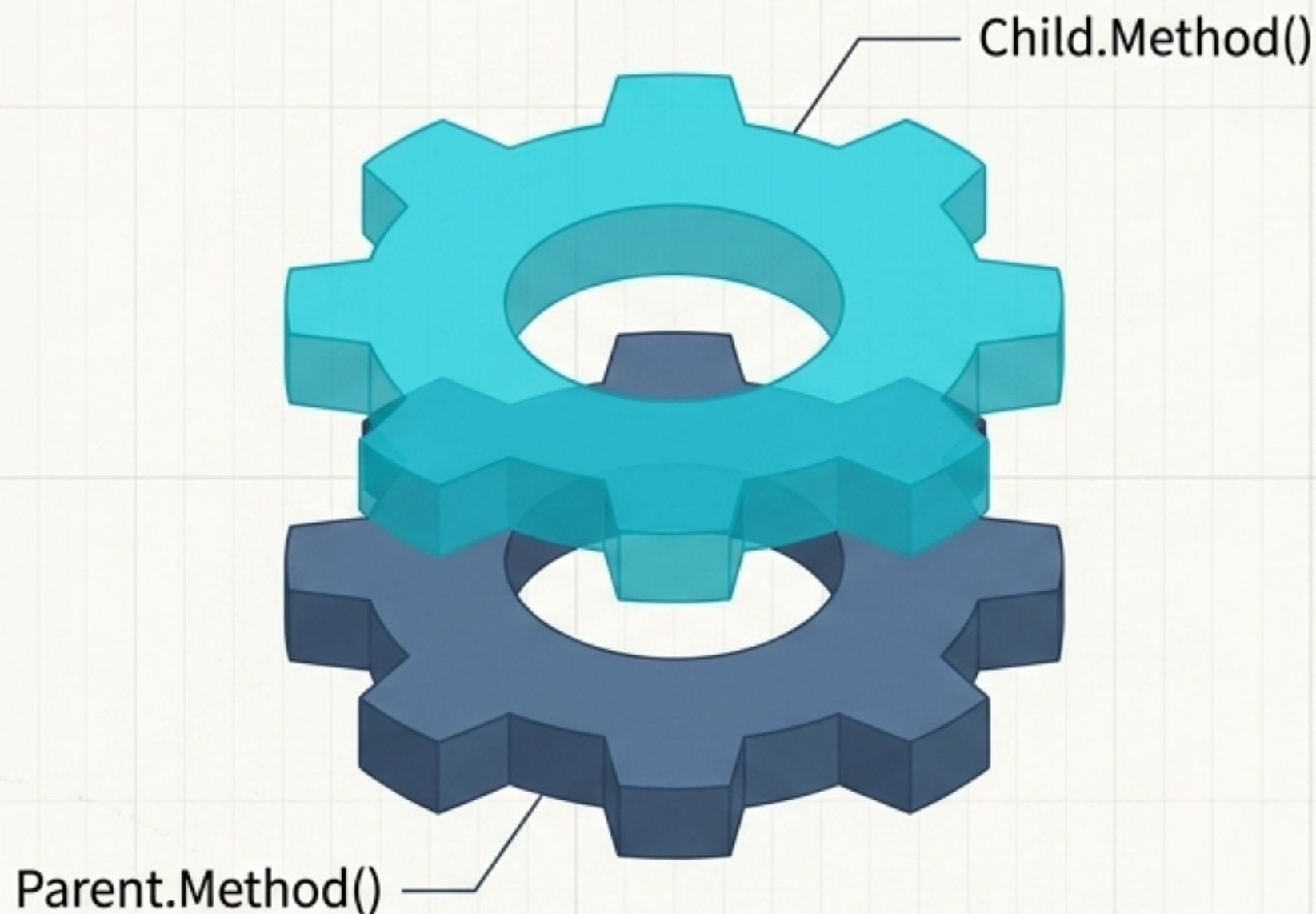
```
class Program
{
    public static int number = 10;

    static void Main(string[] args)
    {
        int number = 20;
        Console.WriteLine(number);
    }
}
```

20

식별자 이름이 겹칠 때, 실행 영역과 가장 가까운 변수가 외부 변수를 가립니다.

메서드 하이딩 (new)



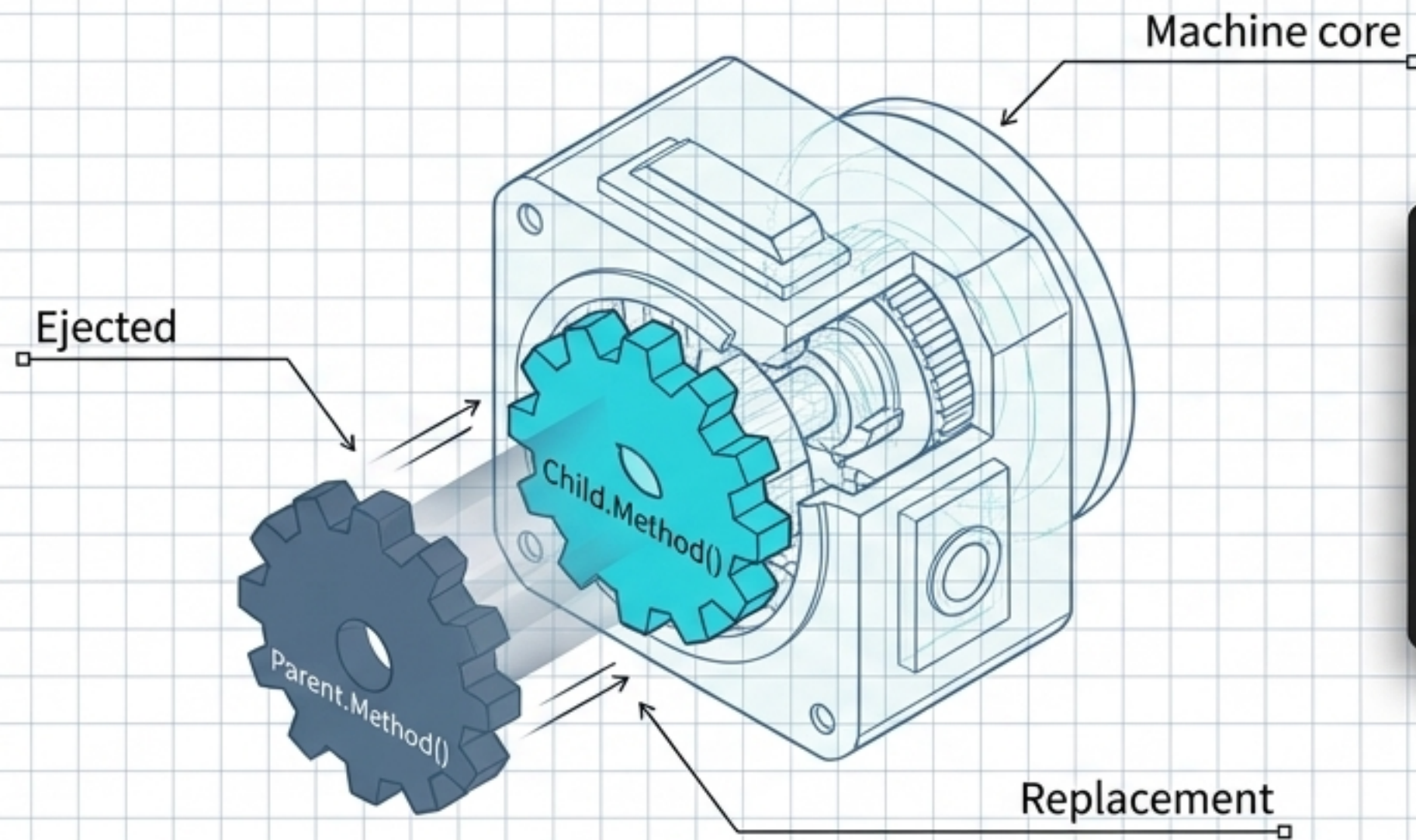
⚠ CS0108: 'Child.Method()'은(는) 상속된 'Parent.Method()' 멤버를 숨깁니다. 숨기려면 new 키워드를 사용하세요.

new 키워드 명시적 사용 (경고 억제)

```
class Child : Parent
{
    public new void Method() { ... }
    ...
}
```

부모의 메서드를 덮어쓰지 않고, 단순히 동일한 이름의 새 멤버를 만들어 가립니다.
부모 타입으로 형변환 시 원본 부모 메서드 실행 가능.

메서드 오버라이딩 (override)



```
public virtual void Method()
```

```
public override void Method()
```

부모의 메서드 작동 방식 자체를 자식의 것으로 근본적으로 덮어씹습니다.
어떤 클래스형으로 접근해도 항상 자식의 메서드가 호출됩니다.

메서드 호출 진단 매트릭스

참조 타입 (변수의 껍데기)	
하이딩 (new 키워드)	부모의 메서드
오버라이딩 (override 키워드)	자식의 메서드

Parent parent = new Child();

Child child = new Child();

자식의 메서드

자식의 메서드

하이딩은 변수의 타입(껍데기)을, 오버라이딩은 생성된 인스턴스(알맹이)를 따릅니다.

절대 제한 (sealed)



```
sealed class Parent  
{  
}
```

컴파일 오류:
'Parent'는 sealed 형식이므로
파생할 수 없습니다.



```
sealed override void Test()  
{  
}
```

컴파일 오류: 'Child.Test()'는
sealed 멤버이므로
오버라이드할 수 없습니다.

클래스에 적용하면 더 이상 상속될 수 없으며,
메서드에 적용하면 오버라이딩의 사슬을 영구적으로 끊습니다.

절대 요구 (abstract)

abstract class



```
Parent parent = new Parent();  
// 오류 발생
```

컴파일 오류:
'Parent'는 abstract 형식이므로
인스턴스화할 수 없습니다.

abstract void

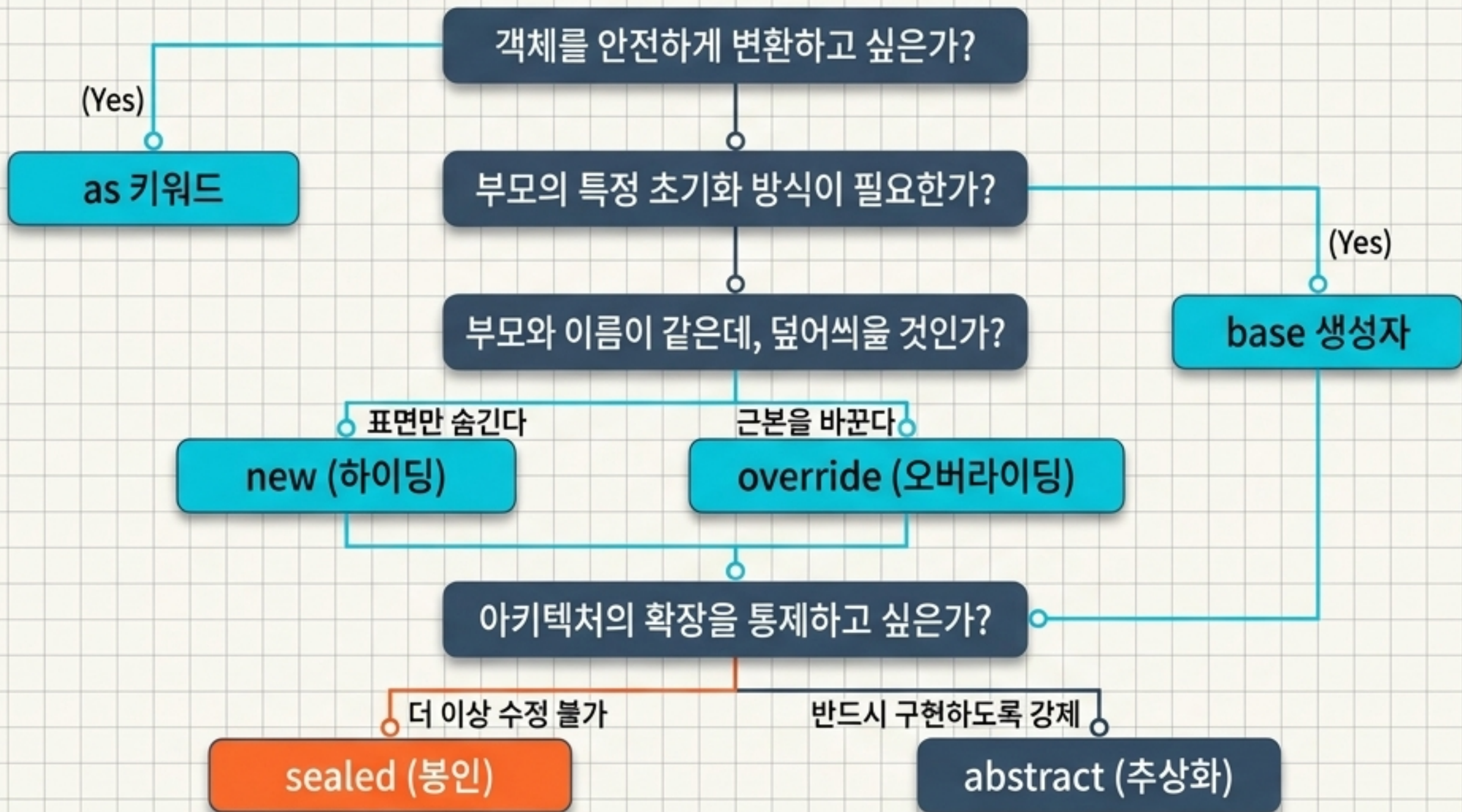


```
class Child : Parent  
{  
    // 오류: 구현 없음  
}
```

컴파일 오류: 'Child'는 상속된
추상 멤버 'Parent.Test()'를
구현하지 않습니다.

자체적으로 인스턴스가 될 수 없으며, 자식 클래스에게 구체적인 구현을 강제하는 설계의 출발점입니다.

객체 지향 설계 의사결정 트리



목적에 맞는 키워드 선택이 견고한 객체 지향 아키텍처를 완성합니다. 통제권은 개발자의 설계도에 있습니다.